Frontend Architectural Patterns

1. Overview

Frontend architectural patterns help structure the interaction between the user interface, application data, and business logic across platforms such as web, mobile, and desktop. By separating concerns and defining clear responsibilities for each layer, these patterns promote scalability, maintainability, and testability.

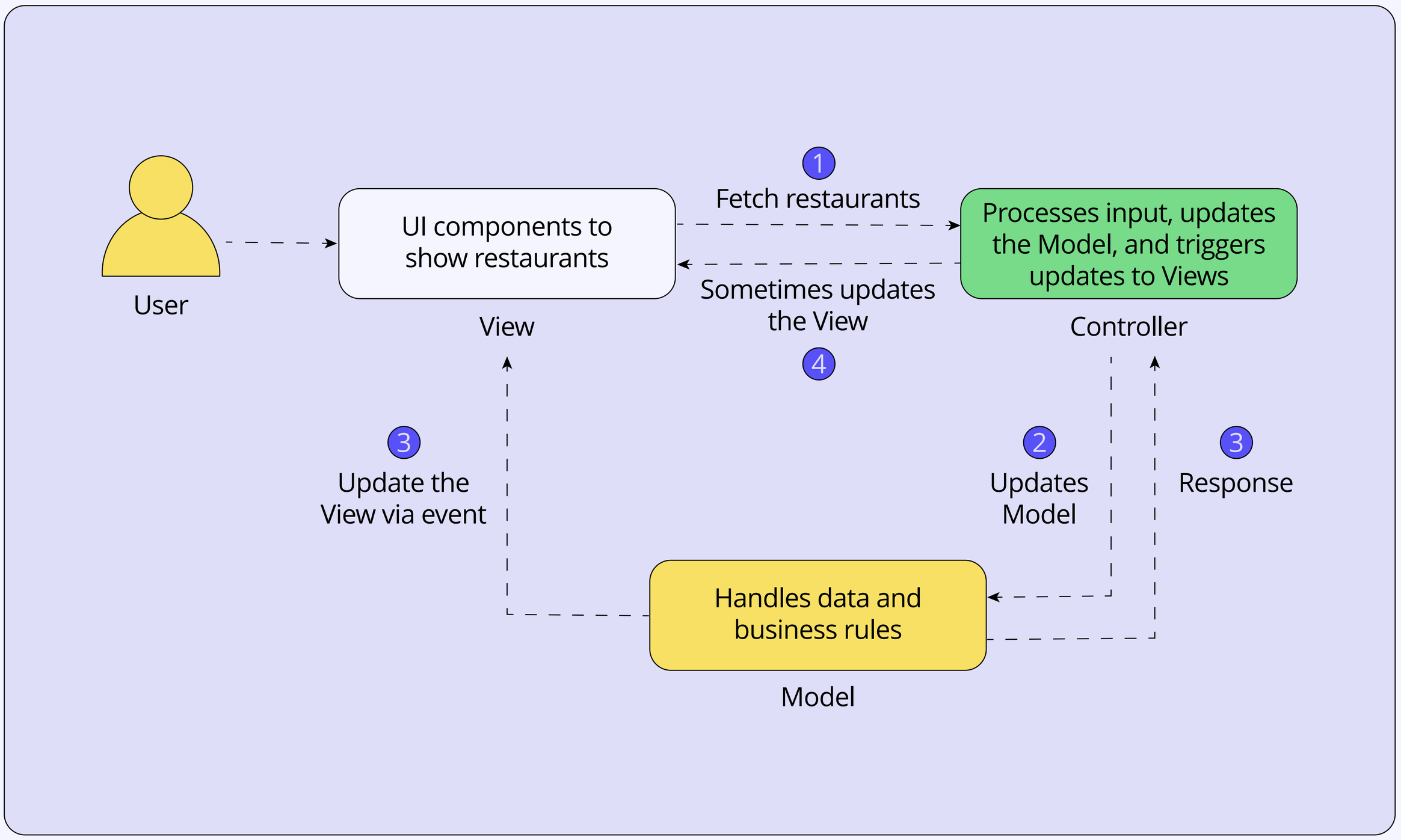
2. Some Key Frontend Architectural Patterns

a. MVC (Model-View-Controller)

The MVC pattern divides an application into three components:

- **Model:** It manages data and business logic.
- **View:** It displays data and UI elements to the user.
- **Controller:** It handles user input, updates the Model, and refreshes the View.

The following illustrations provide an overview of MVC in a food delivery app:



An overview of the MVC architecture pattern

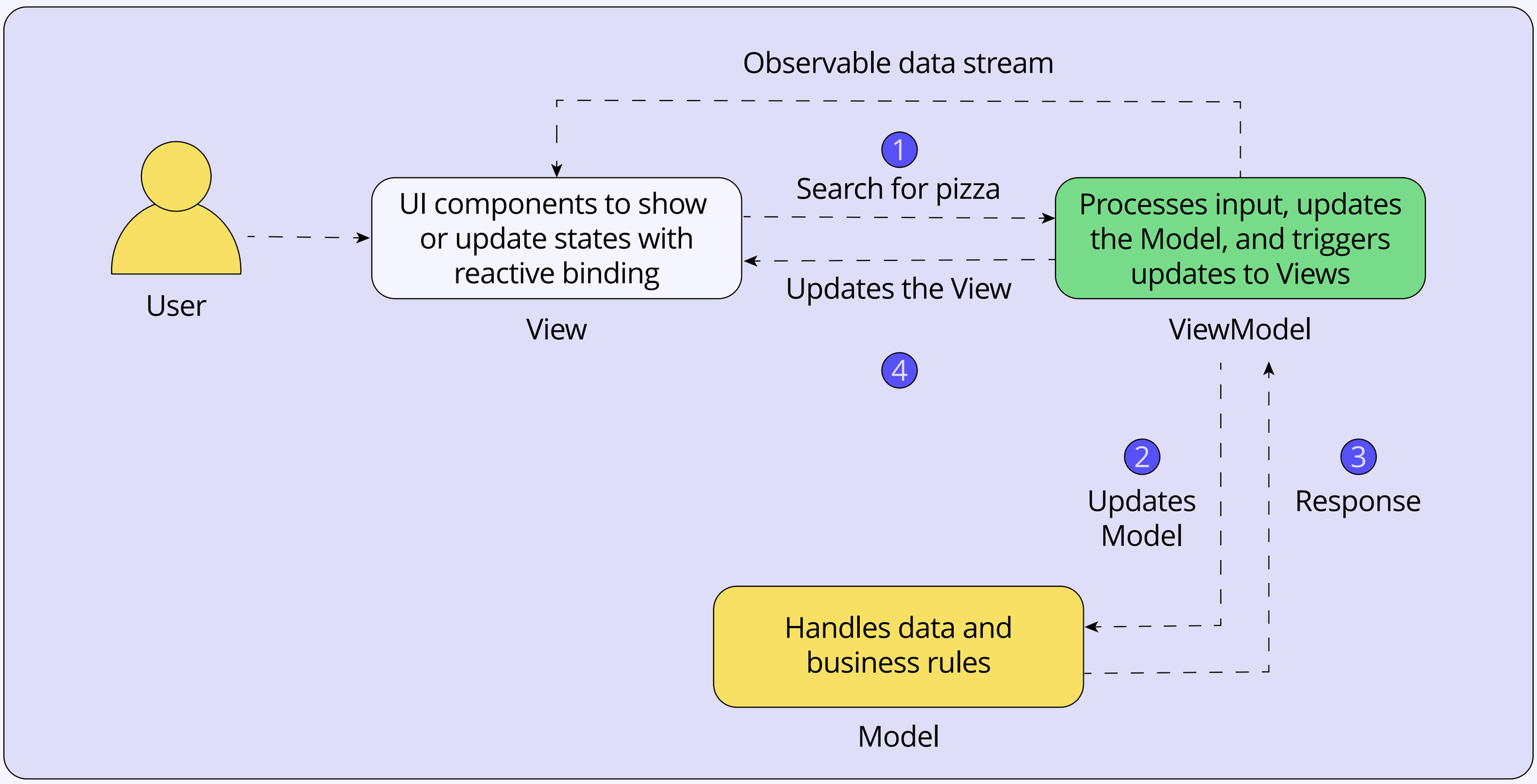
When to Use	Considerations
It is best for simple, small-to-medium apps with clear separation of concerns between data, UI, and logic.	MVC can be limiting for complex, dynamic applications as the Controller can become tightly coupled with the View, making UI updates harder.

b. MVVM (Model-View-ViewModel)

MVVM introduces a ViewModel layer that handles the data binding between the

- **Model:** It manages data and business logic.
- **ViewModel:** It prepares data from the Model for easy consumption by the View and automatically synchronizes changes.
- **View:** It is a passive UI that is updated automatically via data bindings.

The following illustration shows the flow of MVVM in a food delivery app:



An overview of the MVVM pattern

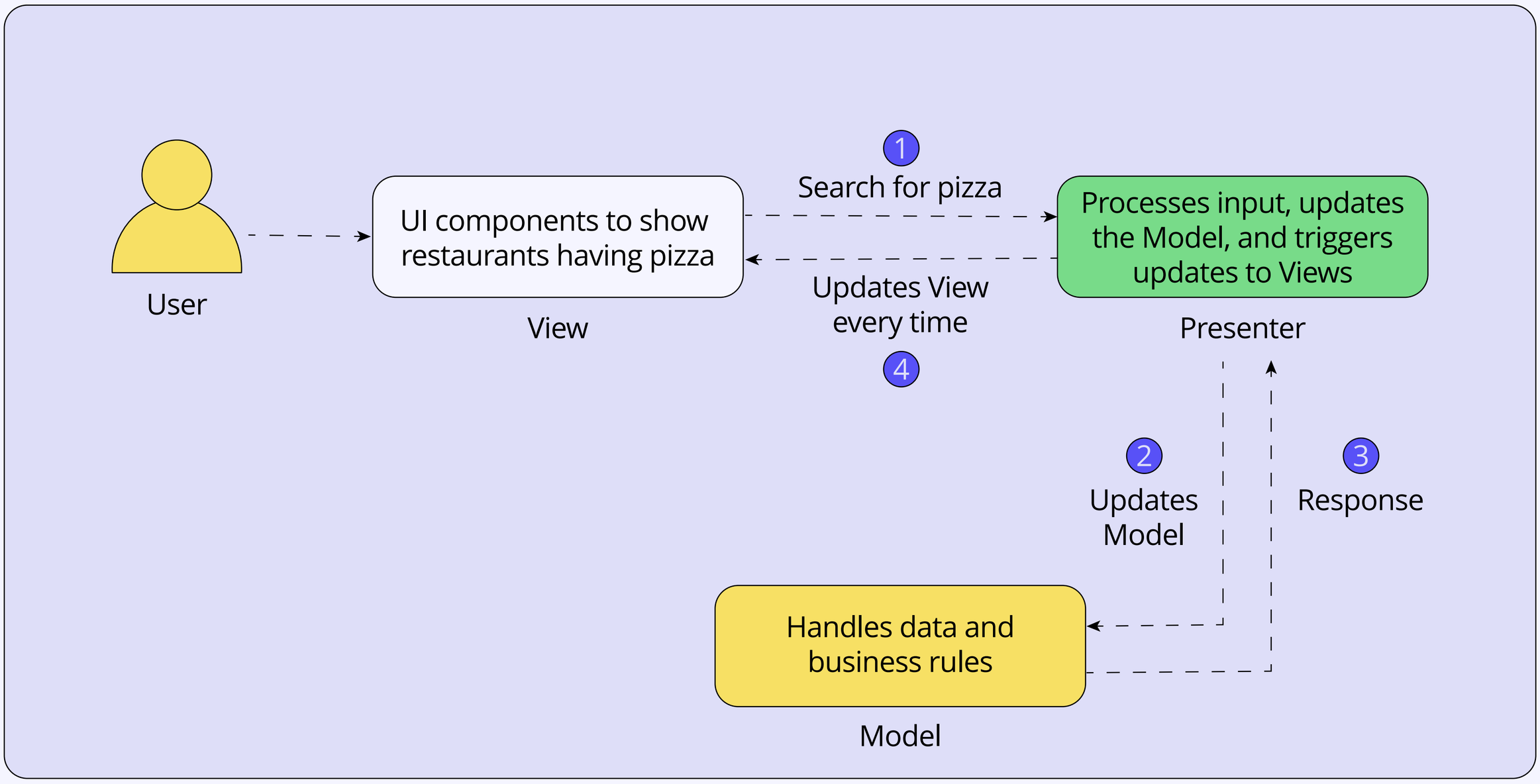
When to Use	Considerations
It is great for interactive, data-driven applications with real-time updates, such as dynamic forms or dashboards.	It can lead to bloated ViewModels, and data binding may reduce control over UI updates.

c. MVP (Model-View-Presenter)

MVP offers clear control over the View and Model through a Presenter that handles all user interactions and updates.

- **Model:** It handles data and business logic.
- **Presenter:** It explicitly manages the logic and updates the View and the Model.
- **View:** It is a passive component that forwards user interactions to the Presenter.

The following illustration shows MVP in a food delivery app:



An overview of the MVP architecture pattern

When to Use	Considerations
It is ideal for complex applications in need of high testability, control over UI updates, and strict separation between UI and logic.	It has more boilerplate and complexity compared to MVC and MVVM, making it overengineered for simple applications.

3. Comparison of MVC, MVVM, and MVP

The following table provides a comparison of each architectural pattern based on different aspects:

Aspect	MVC	MVVM	MVP
Pros	• Clear separation of concerns • Simple and intuitive • Well-suited for small to medium apps	• Two-way data binding reduces boilerplate • Great for reactive UIs • Easy to maintain and scale	• Excellent testability • Explicit control over UI flow • The View is fully decoupled from logic
Cons	• Tight coupling between Controller and View	• Hidden data flows can make debugging harder • Overhead in simple apps • ViewModel can become bloated	• More boilerplate • Manual updates for UI • Slightly more complex structure
Best for	• Simpler apps or those with mostly static UIs	• Interactive, real-time, and component-driven UIs	• Enterprise apps, high test coverage, or where UI must remain passive

4. Conclusion

Each architectural pattern serves a distinct purpose and fits different types of applications. By understanding their pros and cons, you can choose the best approach to ensure your project is well-structured, scalable, and maintainable.

If you want to deepen your knowledge and master frontend System Design, explore the course **Grokking the Frontend System Design Interview** to gain practical insights and hands-on strategies.

